CancerCare AI Platform

Complete Design & Implementation Guide

From Zero to Deployed — Step by Step

Machine Learning Project | LLM-Powered | 100% Free Stack

Table of Contents

Phase 0: Project Overview & Architecture3
Phase 1: Environment Setup4
Phase 2: Project Structure & Git5
Phase 3: Backend — FastAPI Core6
Phase 4: RAG Chatbot Pipeline8
Phase 5: Report Parser10
Phase 6: Diet Recommendation Engine11
Phase 7: ML Models (Risk & NER)12
Phase 8: Symptom & Medication Tracker13
Phase 9: Frontend — React14
Phase 10: Database Setup16
Phase 11: Deployment17
Phase 12: Datasets Guide19
Phase 13: Testing & Validation20
Appendix: API Keys, Env Variables, Disclaimers21

Phase 0: Project Overview & Architecture

What You Are Building

CancerCare AI is a full-stack ML web application that helps cancer patients manage their health journey. It combines LLM-powered chat (RAG), medical report parsing, personalized diet planning, symptom tracking, clinical trial matching, and a caregiver portal — all hosted on free platforms.

Full Tech Stack

Layer	Technology	Hosting / Cost
Frontend	React 18 + Tailwind CSS	Vercel (free)
Backend	FastAPI (Python 3.11)	Render.com (free)
Primary LLM	Gemini 1.5 Flash	Google AI Studio (free)
Fast LLM	Groq + Llama 3.1 70B	Groq Cloud (free)
NLP Models	HuggingFace Transformers	Always free
Embeddings	sentence-transformers	Local / HF Inference
Vector DB	FAISS (local) + Pinecone	Pinecone free tier
Primary DB	Supabase (PostgreSQL)	500MB free
Document DB	MongoDB Atlas	512MB free
Cache	Upstash Redis	10K req/day free
File Storage	Supabase Storage	1GB free
Email	SendGrid	100 emails/day free
CI/CD	GitHub Actions	Free for public repos
Notebooks	Google Colab	Always free

Phase 1: Environment Setup

1.1 Install Prerequisites

Install the following tools in this exact order on your machine.

Python 3.11

- Go to python.org/downloads and download Python 3.11.x
- During install on Windows: check 'Add Python to PATH'
- Verify: open terminal and run

```
python --version
# Expected: Python 3.11.x
```

Node.js 20 LTS

- Go to nodejs.org and download Node.js 20 LTS
- Verify:

```
node --version
# Expected: v20.x.x
npm --version
# Expected: 10.x.x
```

Git

- Download from git-scm.com and install with default options
- Verify:

```
git --version
# Expected: git version 2.x.x
```

- Configure your identity (run once):

```
git config --global user.name "Your Name"
git config --global user.email "your@email.com"
```

VS Code (Recommended Editor)

- Download from code.visualstudio.com
- Install these extensions: Python, Pylance, ESLint, Tailwind CSS IntelliSense, GitLens

1.2 Create Project Folder & Virtual Environment

A virtual environment isolates your Python packages so they don't conflict with other projects.

```
# Step 1: Create your project folder
mkdir cancercare-ai
cd cancercare-ai

# Step 2: Create Python virtual environment
python -m venv venv

# Step 3: Activate it
# On Windows:
venv\Scripts\activate

# On Mac/Linux:
source venv/bin/activate

# You will see (venv) at the start of your terminal line
# This means the environment is active

# Step 4: Upgrade pip
pip install --upgrade pip
```

Important

Every time you open a new terminal to work on this project, you MUST activate the virtual environment first using the activate command above. If you see ModuleNotFoundError, it usually means the venv is not active.

1.3 Install All Python Dependencies

```
pip install fastapi uvicorn[standard] python-multipart
pip install langchain langchain-community langchain-google-genai
pip install faiss-cpu sentence-transformers
pip install pymupdf pytesseract pillow
pip install transformers torch torchvision --index-url
https://download.pytorch.org/whl/cpu
pip install scikit-learn xgboost pandas numpy matplotlib seaborn
pip install supabase pymongo motor redis
pip install python-jose[cryptography] passlib[bcrypt] python-dotenv
pip install httpx requests aiohttp
pip install reportlab apscheduler
pip install groq google-generativeai

# Save to requirements file
pip freeze > requirements.txt
```

Note on torch

Installing torch with --index-url uses the CPU-only version which is smaller (~200MB vs 2GB). This is fine for inference. For GPU training use the default torch install.

Phase 2: Project Structure & Git Setup

2.1 Full Folder Structure

Create this exact folder structure. Each folder's purpose is explained below.

```
cancercare-ai/
├── backend/
|   ├── app/
|   |   ├── __init__.py
|   |   ├── main.py                # FastAPI entry point
|   |   ├── config.py              # Settings & env vars
|   |   ├── database.py            # DB connections
|   |   ├── auth/
|   |   |   ├── __init__.py
|   |   |   ├── router.py          # Login/register endpoints
|   |   |   ├── models.py          # Pydantic schemas
|   |   |   └── utils.py           # JWT helpers
|   |   ├── routes/
|   |   |   ├── chat.py            # Chatbot RAG endpoint
|   |   |   ├── diet.py            # Diet plan generation
|   |   |   ├── reports.py         # PDF upload & parse
|   |   |   ├── symptoms.py        # Symptom tracking
|   |   |   ├── medications.py     # Medication reminders
|   |   |   ├── trials.py          # Clinical trial matcher
|   |   |   └── caregiver.py       # Caregiver portal
|   |   └── ml/
|   |       ├── rag_pipeline.py    # RAG logic
|   |       ├── report_parser.py   # PDF text extraction
|   |       ├── diet_engine.py     # Diet recommendation
|   |       ├── risk_model.py      # Risk scorer
|   |       ├── ner_extractor.py   # BioBERT NER
|   |       └── sentiment.py       # Mood analysis
|   ├── data/
|   |   ├── knowledge_base/        # Medical PDFs for RAG
|   |   └── vector_store/          # FAISS index files
|   ├── models/                    # Saved ML model files (.pkl)
|   ├── tests/
|   |   ├── test_chat.py
|   |   ├── test_diet.py
|   |   └── test_reports.py
|   └── Dockerfile
├── frontend/
|   ├── public/
|   ├── src/
```

```
│   │   ├── pages/
│   │   │   ├── Chatbot.jsx
│   │   │   ├── DietPlan.jsx
│   │   │   ├── Dashboard.jsx
│   │   │   ├── SymptomTracker.jsx
│   │   │   ├── Medications.jsx
│   │   │   ├── ClinicalTrials.jsx
│   │   │   ├── Caregiver.jsx
│   │   │   └── ReportUpload.jsx
│   │   ├── components/
│   │   │   ├── Navbar.jsx
│   │   │   ├── ChatBubble.jsx
│   │   │   ├── DietCard.jsx
│   │   │   ├── SymptomChart.jsx
│   │   │   └── MedicalDisclaimer.jsx
│   │   ├── hooks/
│   │   │   ├── useAuth.js
│   │   │   └── useChat.js
│   │   ├── lib/
│   │   │   └── api.js              # Axios instance
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── package.json
│   └── tailwind.config.js
├── notebooks/
│   ├── 01_eda_tcga.ipynb
│   ├── 02_train_risk_model.ipynb
│   ├── 03_build_rag_knowledge_base.ipynb
│   └── 04_diet_classifier.ipynb
├── .env                            # NEVER commit this file
├── .gitignore
└── README.md
```

2.2 Create the Folders

```
# Run from inside cancercare-ai/
mkdir -p backend/app/auth backend/app/routes backend/app/ml
mkdir -p backend/data/knowledge_base backend/data/vector_store
mkdir -p backend/models backend/tests
mkdir -p frontend/src/pages frontend/src/components
mkdir -p frontend/src/hooks frontend/src/lib
mkdir -p notebooks

# Create empty __init__.py files for Python packages
touch backend/app/__init__.py
```

```
touch backend/app/auth/__init__.py
touch backend/app/routes/__init__.py
touch backend/app/ml/__init__.py
```

2.3 Initialize Git & Push to GitHub

1. Go to github.com and create a new repository called cancercare-ai
2. Make it Public (important for free GitHub Actions CI/CD)
3. Do NOT initialize with README (you will do that locally)
4. Run these commands:

```
# Inside cancercare-ai/ folder
git init
git branch -M main

# Add remote origin (replace YOUR_USERNAME with your GitHub username)
git remote add origin https://github.com/YOUR_USERNAME/cancercare-ai.git
```

2.4 Create .gitignore

This file tells Git which files to NOT track. Create it in the root folder:

```
# Python
venv/
__pycache__/
*.pyc
*.pyo
*.egg-info/
.pytest_cache/

# Environment variables - NEVER commit
.env
.env.local
.env.production

# ML models and data (too large for Git)
backend/models/*.pkl
backend/data/vector_store/
backend/data/knowledge_base/*.pdf

# Node.js
node_modules/
frontend/dist/
frontend/.next/

# OS files
```

```
.DS_Store
Thumbs.db

# IDE
.vscode/settings.json
.idea/
```

2.5 Create .env File

Create a file named .env in the root directory (never commit this):

```
# LLM APIs (all free)
GEMINI_API_KEY=your_gemini_key_here
GROQ_API_KEY=your_groq_key_here
HUGGINGFACE_API_KEY=your_hf_key_here

# Databases
SUPABASE_URL=https://yourproject.supabase.co
SUPABASE_KEY=your_supabase_anon_key
MONGODB_URI=mongodb+srv://user:pass@cluster.mongodb.net/cancercare
UPSTASH_REDIS_URL=rediss://your-upstash-url

# Auth
JWT_SECRET=generate_a_long_random_string_here
JWT_ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60

# Email
SENDGRID_API_KEY=your_sendgrid_key
FROM_EMAIL=noreply@yourdomain.com

# App
ENVIRONMENT=development
FRONTEND_URL=http://localhost:5173
```

2.6 First Git Commit

```
git add .
git commit -m "feat: initial project structure"
git push -u origin main
```

Phase 3: Backend — FastAPI Core

3.1 Config & Settings (backend/app/config.py)

```
from pydantic_settings import BaseSettings
from functools import lru_cache

class Settings(BaseSettings):
    gemini_api_key: str
    groq_api_key: str
    huggingface_api_key: str
    supabase_url: str
    supabase_key: str
    mongodb_uri: str
    jwt_secret: str
    jwt_algorithm: str = 'HS256'
    access_token_expire_minutes: int = 60
    sendgrid_api_key: str
    frontend_url: str = 'http://localhost:5173'

    class Config:
        env_file = '.env'

@lru_cache()
def get_settings():
    return Settings()
```

3.2 Main App Entry Point (backend/app/main.py)

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.config import get_settings
from app.routes import chat, diet, reports, symptoms, medications, trials,
caregiver
from app.auth import router as auth_router

settings = get_settings()

app = FastAPI(
    title='CancerCare AI API',
    description='AI-powered cancer patient support platform',
    version='1.0.0'
)
```

```
# CORS — allow your frontend to call the backend
app.add_middleware(
    CORSMiddleware,
    allow_origins=[settings.frontend_url, 'http://localhost:5173'],
    allow_credentials=True,
    allow_methods=['*'],
    allow_headers=['*'],
)

# Register all route groups
app.include_router(auth_router, prefix='/api/auth', tags=['Authentication'])
app.include_router(chat.router, prefix='/api/chat', tags=['Chatbot'])
app.include_router(diet.router, prefix='/api/diet', tags=['Diet'])
app.include_router(reports.router, prefix='/api/reports', tags=['Reports'])
app.include_router(symptoms.router, prefix='/api/symptoms', tags=['Symptoms'])
app.include_router(medications.router, prefix='/api/medications',
tags=['Medications'])
app.include_router(trials.router, prefix='/api/trials', tags=['Clinical Trials'])
app.include_router(caregiver.router, prefix='/api/caregiver', tags=['Caregiver'])

@app.get('/')
async def root():
    return {'status': 'CancerCare AI is running'}

@app.get('/health')
async def health():
    return {'status': 'healthy'}
```

3.3 Authentication (JWT)

backend/app/auth/utils.py

```
from datetime import datetime, timedelta
from jose import JWTError, jwt
from passlib.context import CryptContext
from app.config import get_settings

settings = get_settings()
pwd_context = CryptContext(schemes=['bcrypt'], deprecated='auto')

def hash_password(password: str) -> str:
    return pwd_context.hash(password)

def verify_password(plain: str, hashed: str) -> bool:
    return pwd_context.verify(plain, hashed)
```

```
def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() +
timedelta(minutes=settings.access_token_expire_minutes)
    to_encode.update({'exp': expire})
    return jwt.encode(to_encode, settings.jwt_secret,
algorithm=settings.jwt_algorithm)


def decode_token(token: str) -> dict:
    try:
        return jwt.decode(token, settings.jwt_secret,
algorithms=[settings.jwt_algorithm])
    except JWTError:
        return None
```

backend/app/auth/router.py — Register & Login Endpoints

```
from fastapi import APIRouter, HTTPException, Depends
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from pydantic import BaseModel, EmailStr
from app.auth.utils import hash_password, verify_password, create_access_token,
decode_token
from app.database import get_supabase

router = APIRouter()
security = HTTPBearer()

class RegisterRequest(BaseModel):
    name: str
    email: EmailStr
    password: str
    cancer_type: str
    role: str = 'patient'  # or 'caregiver'

class LoginRequest(BaseModel):
    email: EmailStr
    password: str

@router.post('/register')
async def register(data: RegisterRequest):
    sb = get_supabase()
    # Check if user exists
    existing = sb.table('users').select('id').eq('email', data.email).execute()
    if existing.data:
        raise HTTPException(status_code=400, detail='Email already registered')
    # Create user
    user = sb.table('users').insert({
```

```
        'name': data.name,
        'email': data.email,
        'password_hash': hash_password(data.password),
        'cancer_type': data.cancer_type,
        'role': data.role
    }).execute()
    token = create_access_token({'sub': user.data[0]['id'], 'role': data.role})
    return {'access_token': token, 'token_type': 'bearer'}

@router.post('/login')
async def login(data: LoginRequest):
    sb = get_supabase()
    user = sb.table('users').select('*').eq('email', data.email).execute()
    if not user.data or not verify_password(data.password,
user.data[0]['password_hash']):
        raise HTTPException(status_code=401, detail='Invalid credentials')
    token = create_access_token({'sub': user.data[0]['id'], 'role':
user.data[0]['role']})
    return {'access_token': token, 'token_type': 'bearer', 'user': user.data[0]}

def get_current_user(credentials: HTTPAuthorizationCredentials =
Depends(security)):
    payload = decode_token(credentials.credentials)
    if not payload:
        raise HTTPException(status_code=401, detail='Invalid or expired token')
    return payload
```

3.4 Run the Backend Locally

```
# Make sure venv is active, then from backend/ folder:
cd backend
uvicorn app.main:app --reload --port 8000

# Open your browser at:
# http://localhost:8000        -> root check
# http://localhost:8000/docs   -> Swagger UI (auto-generated API docs)
# http://localhost:8000/health -> health check
```

Phase 4: RAG Chatbot Pipeline

The chatbot uses Retrieval-Augmented Generation (RAG). It works in 3 stages: (1) Index medical documents into a vector database, (2) When a user asks a question, find the most relevant document chunks, (3) Send those chunks + the question to Gemini/Groq to generate a grounded answer.

4.1 Build the Knowledge Base (run once)

First, download medical PDFs. You will add these to backend/data/knowledge_base/:

- Download cancer patient guides from cancer.gov (free PDFs)
- Use PubMed API to download open-access papers on cancer nutrition and treatment
- Download NCI's Physician Data Query (PDQ) summaries as text files

backend/app/ml/rag_pipeline.py

```
import os
import pickle
from pathlib import Path
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_community.vectorstores import FAISS
from langchain_community.embeddings import HuggingFaceEmbeddings
from langchain_community.document_loaders import PyMuPDFLoader, DirectoryLoader
from langchain.schema import Document

VECTOR_STORE_PATH = Path('data/vector_store')
KNOWLEDGE_BASE_PATH = Path('data/knowledge_base')
EMBEDDING_MODEL = 'sentence-transformers/all-MiniLM-L6-v2'

def get_embeddings():
    return HuggingFaceEmbeddings(model_name=EMBEDDING_MODEL)

def build_knowledge_base():
    """Run this once to index all PDFs into FAISS."""
    print('Loading documents...')
    loader = DirectoryLoader(
        str(KNOWLEDGE_BASE_PATH),
        glob='**/*.pdf',
        loader_cls=PyMuPDFLoader
    )
    docs = loader.load()
    print(f'Loaded {len(docs)} document pages')

    # Split into chunks
```

```
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=1000,
        chunk_overlap=200,
        separators=['\n\n', '\n', '. ', ' ']
    )
    chunks = splitter.split_documents(docs)
    print(f'Created {len(chunks)} chunks')

    # Create vector store
    embeddings = get_embeddings()
    vectorstore = FAISS.from_documents(chunks, embeddings)

    # Save to disk
    VECTOR_STORE_PATH.mkdir(parents=True, exist_ok=True)
    vectorstore.save_local(str(VECTOR_STORE_PATH))
    print('Knowledge base built and saved!')

def load_vectorstore():
    embeddings = get_embeddings()
    return FAISS.load_local(str(VECTOR_STORE_PATH), embeddings,
allow_dangerous_deserialization=True)

def add_patient_report(user_id: str, report_text: str):
    """Add a patient's parsed report to a user-specific FAISS index."""
    doc = Document(page_content=report_text, metadata={'user_id': user_id,
'source': 'patient_report'})
    splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=100)
    chunks = splitter.split_documents([doc])
    embeddings = get_embeddings()
    user_store_path = VECTOR_STORE_PATH / f'user_{user_id}'
    if user_store_path.exists():
        vs = FAISS.load_local(str(user_store_path), embeddings,
allow_dangerous_deserialization=True)
        vs.add_documents(chunks)
    else:
        vs = FAISS.from_documents(chunks, embeddings)
    vs.save_local(str(user_store_path))
```

4.2 Chat Endpoint with RAG

backend/app/routes/chat.py

```
from fastapi import APIRouter, Depends, HTTPException
from pydantic import BaseModel
from typing import List, Optional
from groq import Groq
```

```
from app.config import get_settings
from app.auth.router import get_current_user
from app.ml.rag_pipeline import load_vectorstore, get_embeddings
from pathlib import Path
from langchain_community.vectorstores import FAISS

router = APIRouter()
settings = get_settings()
groq_client = Groq(api_key=settings.groq_api_key)

SYSTEM_PROMPT = '''You are CancerCare AI, a compassionate and knowledgeable health
assistant
specifically designed for cancer patients. You answer questions based on the
provided medical
context. Always:
1. Be empathetic and supportive in tone
2. Cite information from the context when available
3. Remind users that you are not a substitute for professional medical advice
4. If the context does not contain enough information, say so clearly
Do NOT make up medical information.'''

class ChatMessage(BaseModel):
    role: str  # 'user' or 'assistant'
    content: str

class ChatRequest(BaseModel):
    message: str
    history: Optional[List[ChatMessage]] = []
    use_report: Optional[bool] = True

@router.post('/message')
async def chat(request: ChatRequest, current_user: dict =
Depends(get_current_user)):
    user_id = current_user['sub']

    # Load global knowledge base
    global_vs = load_vectorstore()
    global_docs = global_vs.similarity_search(request.message, k=3)

    # Also search user-specific report if it exists
    user_docs = []
    user_store_path = Path(f'data/vector_store/user_{user_id}')
    if request.use_report and user_store_path.exists():
        embeddings = get_embeddings()
        user_vs = FAISS.load_local(str(user_store_path), embeddings,
allow_dangerous_deserialization=True)
        user_docs = user_vs.similarity_search(request.message, k=4)
```

```
    # Build context
    context_parts = []
    if user_docs:
        context_parts.append('=== YOUR MEDICAL REPORT ===')
        context_parts.extend([d.page_content for d in user_docs])
    context_parts.append('=== MEDICAL KNOWLEDGE BASE ===')
    context_parts.extend([d.page_content for d in global_docs])
    context = '\n\n'.join(context_parts)

    # Build messages for LLM
    messages = [{'role': 'system', 'content': SYSTEM_PROMPT}]
    for msg in request.history[-6:]:  # last 3 exchanges
        messages.append({'role': msg.role, 'content': msg.content})
    messages.append({'role': 'user', 'content': f'Context:\n{context}\n\nQuestion:
{request.message}'})

    # Call Groq (Llama 3.1 70B — fast and free)
    response = groq_client.chat.completions.create(
        model='llama-3.1-70b-versatile',
        messages=messages,
        max_tokens=1024,
        temperature=0.3,
    )
    answer = response.choices[0].message.content
    return {'reply': answer, 'sources_used': len(user_docs) + len(global_docs)}
```

Phase 5: Report Parser

5.1 PDF Parsing Logic

backend/app/ml/report_parser.py

```
import fitz  # PyMuPDF
import re
from pathlib import Path
from typing import Dict, List

# Key cancer report fields to extract
EXTRACTION_PATTERNS = {
    'cancer_type': r'(?:diagnosis|cancer type|malignancy)[:\s]+([A-Za-z\s]+)',
    'stage': r'(?:stage|staging)[:\s]+(I{1,3}V?|[1-4][a-c]?)',
    'tumor_markers': r'(?:CA-\d+|PSA|CEA|AFP|HER2)[:\s]+([\d.]+)',
    'hemoglobin': r'(?:hemoglobin|hgb|hb)[:\s]+([\d.]+)',
    'wbc': r'(?:white blood cell|wbc)[:\s]+([\d.]+)',
    'treatment':
r'(?:treatment|chemotherapy|radiation|surgery)[:\s]+([A-Za-z\s,]+)',
    'medications': r'(?:medication|drug|prescribed)[:\s]+([A-Za-z\s,]+)',
}

def extract_text_from_pdf(file_bytes: bytes) -> str:
    """Extract all text from a PDF file."""
    doc = fitz.open(stream=file_bytes, filetype='pdf')
    text_parts = []
    for page in doc:
        text_parts.append(page.get_text())
    return '\n'.join(text_parts)

def extract_key_fields(text: str) -> Dict:
    """Use regex to extract structured data from report text."""
    text_lower = text.lower()
    results = {}
    for field, pattern in EXTRACTION_PATTERNS.items():
        match = re.search(pattern, text_lower, re.IGNORECASE)
        if match:
            results[field] = match.group(1).strip()[:100]
    return results

def parse_report(file_bytes: bytes) -> Dict:
    text = extract_text_from_pdf(file_bytes)
    extracted = extract_key_fields(text)
    return {
```

```
        'raw_text': text[:5000],  # first 5000 chars stored
        'full_text': text,
        'extracted_fields': extracted,
        'page_count': len(fitz.open(stream=file_bytes, filetype='pdf'))
    }
```

5.2 Report Upload Endpoint

backend/app/routes/reports.py

```
from fastapi import APIRouter, UploadFile, File, Depends, HTTPException
from app.auth.router import get_current_user
from app.ml.report_parser import parse_report
from app.ml.rag_pipeline import add_patient_report
from app.database import get_supabase, get_mongo
from datetime import datetime

router = APIRouter()

@router.post('/upload')
async def upload_report(
    file: UploadFile = File(...),
    current_user: dict = Depends(get_current_user)
):
    if file.content_type != 'application/pdf':
        raise HTTPException(status_code=400, detail='Only PDF files are accepted')
    if file.size > 10 * 1024 * 1024:  # 10MB limit
        raise HTTPException(status_code=400, detail='File too large (max 10MB)')

    file_bytes = await file.read()
    user_id = current_user['sub']

    # Parse the report
    parsed = parse_report(file_bytes)

    # Add to user's vector store for RAG
    add_patient_report(user_id, parsed['full_text'])

    # Save parsed data to MongoDB
    mongo = get_mongo()
    mongo['reports'].insert_one({
        'user_id': user_id,
        'filename': file.filename,
        'extracted_fields': parsed['extracted_fields'],
        'raw_text': parsed['raw_text'],
```

```
        'uploaded_at': datetime.utcnow()
    })

    return {'status': 'Report uploaded and indexed', 'extracted':
parsed['extracted_fields']}

@router.get('/my-reports')
async def get_my_reports(current_user: dict = Depends(get_current_user)):
    mongo = get_mongo()
    reports = list(mongo['reports'].find(
        {'user_id': current_user['sub']},
        {'_id': 0, 'full_text': 0}  # exclude large fields
    ).sort('uploaded_at', -1))
    return {'reports': reports}
```

Phase 6: Diet Recommendation Engine

6.1 Diet Generation Logic

backend/app/ml/diet_engine.py

```
import httpx
import google.generativeai as genai
from app.config import get_settings

settings = get_settings()
genai.configure(api_key=settings.gemini_api_key)
model = genai.GenerativeModel('gemini-1.5-flash')

# Cancer type to dietary needs mapping (based on clinical guidelines)
DIETARY_GUIDELINES = {
    'breast': {'avoid': ['alcohol', 'processed meats', 'high fat dairy'],
'emphasize': ['cruciferous vegetables', 'omega-3', 'fiber']},
    'lung': {'avoid': ['beta-carotene supplements'], 'emphasize': ['antioxidants',
'lean protein', 'fruits']},
    'colorectal': {'avoid': ['red meat', 'alcohol', 'processed foods'],
'emphasize': ['fiber', 'whole grains', 'legumes']},
    'default': {'avoid': ['processed foods', 'excess sugar', 'alcohol'],
'emphasize': ['vegetables', 'lean protein', 'whole grains']},
}

async def fetch_usda_foods(query: str) -> list:
    """Fetch nutritional data from USDA FoodData Central (free, no key needed)."""
    async with httpx.AsyncClient() as client:
        r = await client.get(
            'https://api.nal.usda.gov/fdc/v1/foods/search',
            params={'query': query, 'pageSize': 5, 'dataType': 'SR
Legacy,Foundation Food'}
        )
        if r.status_code == 200:
            return r.json().get('foods', [])[:3]
    return []

async def generate_diet_plan(user_profile: dict) -> dict:
    cancer_type = user_profile.get('cancer_type', 'default').lower()
    guidelines = DIETARY_GUIDELINES.get(cancer_type, DIETARY_GUIDELINES['default'])

    prompt = f'''You are a clinical oncology dietitian. Create a detailed 7-day
meal plan for:
- Cancer Type: {user_profile.get('cancer_type', 'unspecified')}
- Treatment Stage: {user_profile.get('stage', 'unspecified')}
```

```
- Fatigue Level: {user_profile.get('fatigue', 5)}/10
- Nausea Level: {user_profile.get('nausea', 3)}/10
- Appetite: {user_profile.get('appetite', 'moderate')}
- Food restrictions: {user_profile.get('restrictions', 'none')}

Dietary guidelines for {cancer_type} cancer:
- Emphasize: {', '.join(guidelines['emphasize'])}
- Avoid: {', '.join(guidelines['avoid'])}

For each day provide: breakfast, mid-morning snack, lunch, afternoon snack, dinner.
Format as JSON with this structure:
{{"day_1": {{"breakfast": {{"meal": "...", "calories": 300, "notes": "..."}},
...}}, ...}}
Include only valid JSON, no explanation.'''

    response = model.generate_content(prompt)
    import json, re
    text = response.text
    # Extract JSON from response
    json_match = re.search(r'\{[\s\S]+\}', text)
    if json_match:
        plan = json.loads(json_match.group())
    else:
        plan = {}
    return {'plan': plan, 'guidelines': guidelines, 'cancer_type': cancer_type}
```

6.2 Diet Tracking Endpoint

backend/app/routes/diet.py

```
from fastapi import APIRouter, Depends
from pydantic import BaseModel
from typing import Optional
from app.auth.router import get_current_user
from app.ml.diet_engine import generate_diet_plan
from app.database import get_supabase
from datetime import datetime

router = APIRouter()

class DietPlanRequest(BaseModel):
    cancer_type: str
    stage: Optional[str] = 'unknown'
    fatigue: Optional[int] = 5
    nausea: Optional[int] = 3
    appetite: Optional[str] = 'moderate'
```

```
    restrictions: Optional[str] = 'none'


class MealLogRequest(BaseModel):
    date: str
    meal_type: str  # breakfast, lunch, dinner, snack
    food_items: list
    calories: Optional[int] = None
    adhered_to_plan: bool = True


@router.post('/generate-plan')
async def generate_plan(request: DietPlanRequest, current_user: dict =
Depends(get_current_user)):
    plan = await generate_diet_plan(request.dict())
    # Save plan to Supabase
    sb = get_supabase()
    sb.table('diet_plans').upsert({
        'user_id': current_user['sub'],
        'plan_data': plan,
        'created_at': datetime.utcnow().isoformat()
    }).execute()
    return plan


@router.post('/log-meal')
async def log_meal(request: MealLogRequest, current_user: dict =
Depends(get_current_user)):
    sb = get_supabase()
    sb.table('meal_logs').insert({
        'user_id': current_user['sub'],
        **request.dict()
    }).execute()
    return {'status': 'Meal logged'}


@router.get('/adherence')
async def get_adherence(current_user: dict = Depends(get_current_user)):
    sb = get_supabase()
    logs = sb.table('meal_logs').select('*').eq('user_id',
current_user['sub']).execute()
    total = len(logs.data)
    adhered = sum(1 for l in logs.data if l.get('adhered_to_plan'))
    return {'total_logs': total, 'adherence_rate': round(adhered/total*100 if total
else 0, 1)}
```

Phase 7: ML Models — Risk Scorer & NER

7.1 Training the Risk Model (Google Colab)

Open Google Colab at colab.research.google.com and create a new notebook called 02_train_risk_model.ipynb. Run each cell in order:

```
# Cell 1: Install dependencies
!pip install pandas scikit-learn xgboost matplotlib seaborn

# Cell 2: Download UCI Breast Cancer dataset (built into sklearn)
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, roc_auc_score
import xgboost as xgb
import pickle, pandas as pd

data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target
print(f'Dataset: {X.shape[0]} samples, {X.shape[1]} features')

# Cell 3: Train/test split and scaling
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Cell 4: Train XGBoost
model = xgb.XGBClassifier(n_estimators=200, max_depth=4, learning_rate=0.05,
random_state=42)
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
print(classification_report(y_test, y_pred, target_names=['Malignant', 'Benign']))
print(f'ROC-AUC: {roc_auc_score(y_test,
model.predict_proba(X_test_scaled)[:,1]):.3f}')

# Cell 5: Save model
with open('risk_model.pkl', 'wb') as f: pickle.dump(model, f)
with open('risk_scaler.pkl', 'wb') as f: pickle.dump(scaler, f)
print('Models saved — download risk_model.pkl and risk_scaler.pkl')
```

After training, download both .pkl files and place them in backend/models/

7.2 BioBERT NER for Report Extraction

backend/app/ml/ner_extractor.py

```
from transformers import pipeline
from functools import lru_cache

@lru_cache(maxsize=1)  # Load model once and cache it
def load_ner_pipeline():
    return pipeline(
        'ner',
        model='allenai/scibert_scivocab_uncased',
        aggregation_strategy='simple'
    )

def extract_medical_entities(text: str) -> dict:
    """Extract medical named entities from clinical text."""
    ner = load_ner_pipeline()
    # Process in chunks to avoid token limit
    max_length = 500
    words = text.split()
    chunks = [' '.join(words[i:i+max_length]) for i in range(0, len(words),
max_length)]

    all_entities = []
    for chunk in chunks[:5]:  # limit to 5 chunks
        entities = ner(chunk)
        all_entities.extend(entities)

    # Group by entity type
    grouped = {}
    for ent in all_entities:
        if ent['score'] > 0.8:  # confidence threshold
            entity_type = ent['entity_group']
            if entity_type not in grouped:
                grouped[entity_type] = []
            word = ent['word'].replace('##', '')
            if word not in grouped[entity_type]:
                grouped[entity_type].append(word)
    return grouped
```

7.3 Sentiment Analyzer for Mood Journal

backend/app/ml/sentiment.py

```
from transformers import pipeline
```

```
from functools import lru_cache


@lru_cache(maxsize=1)
def load_sentiment():
    return pipeline('sentiment-analysis',
model='distilbert-base-uncased-finetuned-sst-2-english')


def analyze_mood(text: str) -> dict:
    """Analyze sentiment of a journal entry."""
    if not text or len(text.strip()) < 10:
        return {'label': 'NEUTRAL', 'score': 0.5}
    sentiment = load_sentiment()
    result = sentiment(text[:512])[0]
    return {
        'label': result['label'],   # POSITIVE or NEGATIVE
        'score': round(result['score'], 3),
        'mood_score': round(result['score'] if result['label'] == 'POSITIVE' else 1
- result['score'], 3)
    }
```

Phase 8: Symptom & Medication Tracker

8.1 Symptom Logging

backend/app/routes/symptoms.py

```
from fastapi import APIRouter, Depends
from pydantic import BaseModel
from typing import Optional
from app.auth.router import get_current_user
from app.ml.sentiment import analyze_mood
from app.database import get_mongo
from datetime import datetime, timedelta

router = APIRouter()

class SymptomLog(BaseModel):
    date: str
    fatigue: int        # 1-10
    nausea: int         # 1-10
    pain: int           # 1-10
    appetite: int       # 1-10
    mood: int           # 1-10
    sleep_hours: float
    journal_text: Optional[str] = ''

@router.post('/log')
async def log_symptoms(log: SymptomLog, current_user: dict =
Depends(get_current_user)):
    mongo = get_mongo()
    user_id = current_user['sub']

    # Analyze journal mood
    sentiment = analyze_mood(log.journal_text) if log.journal_text else {}

    # Check for consistently low mood (alert after 3 days)
    three_days_ago = datetime.utcnow() - timedelta(days=3)
    recent = list(mongo['symptoms'].find({'user_id': user_id, 'logged_at': {'$gte':
three_days_ago}}))
    low_mood_alert = len(recent) >= 2 and all(r.get('mood', 5) <= 3 for r in
recent) and log.mood <= 3

    mongo['symptoms'].insert_one({
        'user_id': user_id,
        **log.dict(),
        'sentiment': sentiment,
```

```
        'logged_at': datetime.utcnow()
    })
    response = {'status': 'Symptom log saved'}
    if low_mood_alert:
        response['alert'] = 'We noticed you have been feeling low for a few days.
Please consider speaking with your care team or a counselor. You are not alone.'
    return response

@router.get('/trends')
async def get_trends(days: int = 14, current_user: dict =
Depends(get_current_user)):
    mongo = get_mongo()
    since = datetime.utcnow() - timedelta(days=days)
    logs = list(mongo['symptoms'].find(
        {'user_id': current_user['sub'], 'logged_at': {'$gte': since}},
        {'_id': 0}
    ).sort('logged_at', 1))
    return {'trends': logs, 'days': days}
```

8.2 Clinical Trial Matcher

backend/app/routes/trials.py

```
from fastapi import APIRouter, Depends
from app.auth.router import get_current_user
from app.database import get_supabase
import httpx

router = APIRouter()

@router.get('/search')
async def search_trials(
    cancer_type: str = '',
    status: str = 'RECRUITING',
    current_user: dict = Depends(get_current_user)
):
    # ClinicalTrials.gov API v2 — completely free
    async with httpx.AsyncClient() as client:
        r = await client.get(
            'https://clinicaltrials.gov/api/v2/studies',
            params={
                'query.cond': f'cancer {cancer_type}',
                'filter.overallStatus': status,
                'fields':
'NCTId,BriefTitle,Condition,Phase,LocationCountry,BriefSummary',
                'pageSize': 10
```

```
            },
            timeout=15.0
        )
    if r.status_code != 200:
        return {'trials': [], 'error': 'Could not reach ClinicalTrials.gov'}
    data = r.json()
    trials = []
    for study in data.get('studies', []):
        p = study.get('protocolSection', {})
        trials.append({
            'id': p.get('identificationModule', {}).get('nctId'),
            'title': p.get('identificationModule', {}).get('briefTitle'),
            'phase': p.get('designModule', {}).get('phases', ['N/A']),
            'summary': p.get('descriptionModule', {}).get('briefSummary',
'')[:300],
            'countries': p.get('contactsLocationsModule', {}).get('locations', []),
        })
    return {'trials': trials, 'total': data.get('totalCount', 0)}
```

Phase 9: Frontend — React

9.1 Initialize React App

```
# From cancercare-ai/ root folder
cd frontend
npm create vite@latest . -- --template react

# When prompted: select React, then JavaScript

# Install dependencies
npm install
npm install axios react-router-dom recharts
npm install @headlessui/react @heroicons/react
npm install react-markdown react-hot-toast

# Install Tailwind CSS
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

9.2 Configure Tailwind (tailwind.config.js)

```
/** @type {import('tailwindcss').Config} */
export default {
  content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],
  theme: {
    extend: {
      colors: {
        teal: { 50: '#f0fdfa', 500: '#14b8a6', 600: '#0d9488', 700: '#0f766e' },
        cancer: { light: '#e0f2fe', mid: '#0ea5e9', dark: '#0369a1' }
      }
    }
  },
  plugins: []
}
```

9.3 API Client (src/lib/api.js)

```
import axios from 'axios'

const API_BASE = import.meta.env.VITE_API_URL || 'http://localhost:8000'
```

```
const api = axios.create({ baseURL: `${API_BASE}/api` })

// Auto-attach JWT token to every request
api.interceptors.request.use(config => {
  const token = localStorage.getItem('token')
  if (token) config.headers.Authorization = `Bearer ${token}`
  return config
})

// Redirect to login on 401
api.interceptors.response.use(
  res => res,
  err => {
    if (err.response?.status === 401) {
      localStorage.removeItem('token')
      window.location.href = '/login'
    }
    return Promise.reject(err)
  }
)

export const chatAPI = {
  sendMessage: (message, history) => api.post('/chat/message', { message, history
}),
}
export const dietAPI = {
  generatePlan: (profile) => api.post('/diet/generate-plan', profile),
  logMeal: (data) => api.post('/diet/log-meal', data),
  getAdherence: () => api.get('/diet/adherence'),
}
export const reportsAPI = {
  uploadReport: (file) => { const fd = new FormData(); fd.append('file', file);
return api.post('/reports/upload', fd) },
  getReports: () => api.get('/reports/my-reports'),
}
export const symptomsAPI = {
  logSymptoms: (data) => api.post('/symptoms/log', data),
  getTrends: (days) => api.get(`/symptoms/trends?days=${days}`),
}
export const trialsAPI = {
  searchTrials: (cancerType) =>
api.get(`/trials/search?cancer_type=${cancerType}`),
}
export default api
```

9.4 Chatbot Page (src/pages/Chatbot.jsx)

```
import { useState, useRef, useEffect } from 'react'
import { chatAPI } from '../lib/api'
import ReactMarkdown from 'react-markdown'

export default function Chatbot() {
  const [messages, setMessages] = useState([
    { role: 'assistant', content: 'Hello! I am CancerCare AI. I can answer
questions about your reports, treatment, diet, and more. What would you like to
know?' }
  ])
  const [input, setInput] = useState('')
  const [loading, setLoading] = useState(false)
  const bottomRef = useRef(null)

  useEffect(() => { bottomRef.current?.scrollIntoView({ behavior: 'smooth' }) },
[messages])

  const sendMessage = async () => {
    if (!input.trim() || loading) return
    const userMsg = { role: 'user', content: input }
    const history = messages.slice(-6)
    setMessages(prev => [...prev, userMsg])
    setInput('')
    setLoading(true)
    try {
      const { data } = await chatAPI.sendMessage(input, history)
      setMessages(prev => [...prev, { role: 'assistant', content: data.reply }])
    } catch {
      setMessages(prev => [...prev, { role: 'assistant', content: 'Sorry, I
encountered an error. Please try again.' }])
    } finally { setLoading(false) }
  }

  return (
    <div className='flex flex-col h-screen max-w-3xl mx-auto p-4'>
      <div className='bg-teal-600 text-white p-4 rounded-t-xl'>
        <h1 className='text-xl font-bold'>CancerCare AI Assistant</h1>
        <p className='text-sm opacity-80'>Powered by your medical reports + medical
knowledge base</p>
      </div>
      {/* Disclaimer */}
      <div className='bg-amber-50 border border-amber-200 p-3 text-xs
text-amber-700'>
        This AI assistant provides general information only and is not a substitute
for professional medical advice.
      </div>
      {/* Messages */}
      <div className='flex-1 overflow-y-auto p-4 space-y-4 bg-gray-50'>
```

```
        {messages.map((msg, i) => (
          <div key={i} className={`flex ${msg.role === 'user' ? 'justify-end' : 'justify-start'}`}>
            <div className={`max-w-[80%] p-3 rounded-xl text-sm ${msg.role === 'user' ? 'bg-teal-600 text-white' : 'bg-white border border-gray-200 text-gray-700'}`}>
              <ReactMarkdown>{msg.content}</ReactMarkdown>
            </div>
          </div>
        ))}
        {loading && <div className='flex justify-start'><div className='bg-white border border-gray-200 p-3 rounded-xl text-sm text-gray-500'>Thinking...</div></div>}
        <div ref={bottomRef} />
      </div>
      {/* Input */}
      <div className='flex gap-2 p-4 bg-white border-t'>
        <input value={input} onChange={e => setInput(e.target.value)} onKeyDown={e => e.key === 'Enter' && sendMessage()}
          placeholder='Ask about your treatment, diet, symptoms...'
          className='flex-1 border border-gray-300 rounded-xl px-4 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-teal-500' />
        <button onClick={sendMessage} disabled={loading}
          className='bg-teal-600 text-white px-4 py-2 rounded-xl text-sm hover:bg-teal-700 disabled:opacity-50'>Send</button>
      </div>
    </div>
  )
}
```

9.5 Symptom Dashboard with Charts (src/pages/Dashboard.jsx)

```
import { useEffect, useState } from 'react'
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, Legend, ResponsiveContainer } from 'recharts'
import { symptomsAPI } from '../lib/api'

export default function Dashboard() {
  const [data, setData] = useState([])
  const [days, setDays] = useState(14)

  useEffect(() => {
    symptomsAPI.getTrends(days).then(res => setData(res.data.trends))
  }, [days])

  return (
    <div className='max-w-4xl mx-auto p-6'>
      <h1 className='text-2xl font-bold text-gray-800 mb-6'>Health Dashboard</h1>
```

```
      <div className='flex gap-2 mb-4'>
        {[7, 14, 30].map(d => (
          <button key={d} onClick={() => setDays(d)}
            className={`px-4 py-1 rounded-full text-sm ${days === d ? 'bg-teal-600
text-white' : 'bg-gray-100 text-gray-600'}`}>
            {d} days
          </button>
        ))}
      </div>
      <div className='bg-white rounded-xl border border-gray-200 p-4'>
        <h2 className='font-semibold text-gray-700 mb-4'>Symptom Trends</h2>
        <ResponsiveContainer width='100%' height={300}>
          <LineChart data={data}>
            <CartesianGrid strokeDasharray='3 3' />
            <XAxis dataKey='date' tick={{ fontSize: 11 }} />
            <YAxis domain={[0, 10]} tick={{ fontSize: 11 }} />
            <Tooltip />
            <Legend />
            <Line type='monotone' dataKey='fatigue' stroke='#ef4444'
strokeWidth={2} dot={false} />
            <Line type='monotone' dataKey='mood' stroke='#10b981' strokeWidth={2}
dot={false} />
            <Line type='monotone' dataKey='pain' stroke='#f59e0b' strokeWidth={2}
dot={false} />
            <Line type='monotone' dataKey='nausea' stroke='#8b5cf6' strokeWidth={2}
dot={false} />
          </LineChart>
        </ResponsiveContainer>
      </div>
    </div>
  )
}
```

9.6 Run Frontend Locally

```
# Inside frontend/ folder
npm run dev

# Open browser at http://localhost:5173
# The frontend will auto-reload on file changes

# Make sure backend is also running at localhost:8000
```

Phase 10: Database Setup

10.1 Supabase (PostgreSQL) — Free 500MB

5. Go to supabase.com and click 'Start your project'
6. Sign up with GitHub
7. Click 'New Project', name it cancercare
8. Copy the Project URL and anon key to your .env file
9. Go to the SQL Editor tab and run this schema:

```
-- Users table
CREATE TABLE users (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  name TEXT NOT NULL,
  email TEXT UNIQUE NOT NULL,
  password_hash TEXT NOT NULL,
  cancer_type TEXT,
  role TEXT DEFAULT 'patient',
  caregiver_for UUID REFERENCES users(id),
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Diet plans
CREATE TABLE diet_plans (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  plan_data JSONB,
  created_at TIMESTAMPTZ DEFAULT NOW()
);

-- Meal logs
CREATE TABLE meal_logs (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  date DATE NOT NULL,
  meal_type TEXT,
  food_items JSONB,
  calories INTEGER,
  adhered_to_plan BOOLEAN DEFAULT true,
  logged_at TIMESTAMPTZ DEFAULT NOW()
);

-- Medications
CREATE TABLE medications (
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,
```

```
  user_id UUID REFERENCES users(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  dosage TEXT,
  frequency TEXT,
  times TEXT[],
  start_date DATE,
  active BOOLEAN DEFAULT true
);

-- Row Level Security (patients see only their own data)
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
ALTER TABLE diet_plans ENABLE ROW LEVEL SECURITY;
ALTER TABLE meal_logs ENABLE ROW LEVEL SECURITY;
ALTER TABLE medications ENABLE ROW LEVEL SECURITY;
```

10.2 MongoDB Atlas — Free 512MB

10. Go to mongodb.com/cloud/atlas and sign up for free
11. Create a new project called CancerCare
12. Create a free M0 cluster (512MB)
13. In Security > Database Access: create a user with readWrite permissions
14. In Security > Network Access: add 0.0.0.0/0 (allow all IPs)
15. Click Connect > Connect your application > copy the connection string
16. Paste it into .env as MONGODB_URI (replace <password> with your actual password)

MongoDB will store: symptom logs, chat history, uploaded report metadata.

10.3 Database Connection Module

backend/app/database.py

```
from supabase import create_client
from pymongo import MongoClient
from functools import lru_cache
from app.config import get_settings

settings = get_settings()

@lru_cache(maxsize=1)
def get_supabase():
    return create_client(settings.supabase_url, settings.supabase_key)

@lru_cache(maxsize=1)
def get_mongo():
    client = MongoClient(settings.mongodb_uri)
```

```
    return client['cancercare']
```

Phase 11: Deployment (Both Free)

11.1 Deploy Backend to Render.com

17. Create a Dockerfile in backend/:

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

18. Commit and push to GitHub:

```
git add backend/Dockerfile
git commit -m "feat: add dockerfile for render deployment"
git push
```

19. Go to render.com and sign up with GitHub
20. Click New > Web Service
21. Connect your GitHub repository
22. Configure the service:

- Root Directory: backend
- Runtime: Docker
- Plan: Free

23. Scroll to Environment Variables and add ALL your .env variables one by one
24. Click Create Web Service — Render will build and deploy automatically
25. Your backend URL will be: https://cancercare-ai-backend.onrender.com

> **Free Tier Limitation**
> Render's free tier spins down after 15 minutes of inactivity. The first request after a cold start takes ~30 seconds. For a portfolio/student project this is fine. To prevent it, use Render's cron job to ping /health every 14 minutes.

11.2 Deploy Frontend to Vercel

26. Create frontend/.env.production:

```
VITE_API_URL=https://cancercare-ai-backend.onrender.com
```

27. Commit and push:

```
git add frontend/.env.production
git commit -m "feat: add production api url"
git push
```

28. Go to vercel.com and sign up with GitHub
29. Click Add New > Project

30. Import your cancercare-ai repository
31. Set Framework Preset to Vite
32. Set Root Directory to frontend
33. Add environment variable: VITE_API_URL = your Render URL
34. Click Deploy
35. Your app will be live at: https://cancercare-ai.vercel.app

11.3 GitHub Actions CI/CD

Create .github/workflows/deploy.yml to auto-test on every push:

```
name: CI
on: [push, pull_request]
jobs:
  test-backend:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.11' }
      - run: pip install -r backend/requirements.txt
      - run: cd backend && python -m pytest tests/ -v
        env:
          GEMINI_API_KEY: ${{ secrets.GEMINI_API_KEY }}
          GROQ_API_KEY: ${{ secrets.GROQ_API_KEY }}
          SUPABASE_URL: ${{ secrets.SUPABASE_URL }}
          SUPABASE_KEY: ${{ secrets.SUPABASE_KEY }}
          MONGODB_URI: ${{ secrets.MONGODB_URI }}
          JWT_SECRET: ${{ secrets.JWT_SECRET }}
```

Add your secrets in GitHub > Repository > Settings > Secrets and variables > Actions.

Phase 12: Datasets Guide

All datasets below are free, publicly available, and no institutional approval is needed for student projects.

Dataset	Source	Usage in Project
UCI Breast Cancer Wisconsin	sklearn.datasets.load_breast_cancer()	Risk model training — instant download via Python
TCGA Clinical Data	portal.gdc.cancer.gov	Cancer type, survival, treatment — requires free login
SEER Database	seer.cancer.gov/data/	Survival statistics — register for free access
PubMed Open Access	pubmed.ncbi.nlm.nih.gov/	Cancer papers for RAG — use Entrez API
NCI PDQ Summaries	cancer.gov/publications/pdq	Patient guides — download as free PDFs
USDA FoodData	fdc.nal.usda.gov	Nutrition data — free REST API, no key needed
Open Food Facts	world.openfoodfacts.org	Food products DB — free download as CSV
ClinicalTrials.gov	clinicaltrials.gov/api/v2	Live trial data — free REST API
Kaggle Cancer NLP	kaggle.com/datasets	Search 'cancer NLP dataset' — free with account
MIMIC-III (optional)	physionet.org/content/mimiciii	Clinical notes — requires credentials

12.1 Downloading PubMed Articles for RAG (Python script)

```
# Run this in a Jupyter notebook or script to build your knowledge base
# pip install biopython

from Bio import Entrez
import time

Entrez.email = 'your@email.com'  # required by NCBI

def download_pubmed_articles(query: str, max_articles: int = 50, output_dir: str =
'data/knowledge_base'):
    import os
    os.makedirs(output_dir, exist_ok=True)
```

```
    # Search PubMed
    handle = Entrez.esearch(db='pubmed', term=query, retmax=max_articles)
    record = Entrez.read(handle)
    ids = record['IdList']
    print(f'Found {len(ids)} articles')

    # Fetch abstracts
    for pmid in ids:
        handle = Entrez.efetch(db='pubmed', id=pmid, rettype='abstract',
retmode='text')
        content = handle.read()
        with open(f'{output_dir}/pubmed_{pmid}.txt', 'w') as f:
            f.write(content)
        time.sleep(0.5)  # be polite to NCBI servers
    print(f'Downloaded {len(ids)} articles to {output_dir}')

# Usage:
download_pubmed_articles('cancer patient nutrition chemotherapy', max_articles=50)
download_pubmed_articles('cancer pain management palliative care', max_articles=30)
download_pubmed_articles('cancer fatigue treatment guidelines', max_articles=30)
```

Phase 13: Testing & Validation

13.1 Backend Tests

backend/tests/test_chat.py

```
import pytest
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_health_check():
    response = client.get('/health')
    assert response.status_code == 200
    assert response.json()['status'] == 'healthy'

def test_register_and_login():
    # Register
    reg = client.post('/api/auth/register', json={
        'name': 'Test Patient',
        'email': 'test@example.com',
        'password': 'TestPass123!',
        'cancer_type': 'breast'
    })
    # May fail if user already exists — that's ok
    # Login
    login = client.post('/api/auth/login', json={
        'email': 'test@example.com',
        'password': 'TestPass123!'
    })
    assert login.status_code == 200
    assert 'access_token' in login.json()

def test_chat_requires_auth():
    response = client.post('/api/chat/message', json={'message': 'hello'})
    assert response.status_code == 403  # no auth token
```

13.2 Run Tests

```
# From backend/ folder with venv active
cd backend
python -m pytest tests/ -v
```

```
# Run specific test
python -m pytest tests/test_chat.py -v

# Run with coverage report
pip install pytest-cov
python -m pytest tests/ --cov=app --cov-report=term-missing
```

13.3 Manual API Testing with Swagger

FastAPI automatically generates interactive documentation at http://localhost:8000/docs. Use it to test every endpoint manually during development. You can authorize with your JWT token by clicking the Authorize button and entering Bearer <your_token>.

Appendix: API Keys, Env Setup & Disclaimers

How to Get Each Free API Key

Service	Where to Get Key	Notes
Gemini API	aistudio.google.com/app/apikey	Click 'Create API Key'. Free up to 15 RPM, 1M tokens/day
Groq API	console.groq.com/keys	Sign up, create key. Free 6000 RPM on Llama 3.1 70B
HuggingFace	huggingface.co/settings/tokens	Create 'read' token. Needed for gated models
Supabase	app.supabase.com > Project Settings > API	Copy URL and anon key
MongoDB Atlas	cloud.mongodb.com > Connect	Copy the connection string URI
SendGrid	app.sendgrid.com/settings/api_keys	Free 100 emails/day. Verify your sender domain
Pinecone (optional)	app.pinecone.io/organizations	Free 1 index. Alternative to FAISS for cloud vector search
Upstash Redis	console.upstash.com	Free database. Copy REST URL and token

Medical Disclaimer — Required on Every Page

IMPORTANT: Medical Disclaimer

CancerCare AI is an educational tool designed to provide general health information and support for cancer patients. It does NOT constitute medical advice, diagnosis, or treatment recommendations. All information provided by this platform must be discussed with a qualified healthcare professional, specifically a licensed oncologist or treating physician, before acting on it. Cancer treatment decisions are highly individualized and must be made in consultation with your medical team. In case of a medical emergency, call emergency services immediately.

Project Milestones — Suggested Timeline

Timeline	Phases	Deliverable
Week 1	Phase 1-2	Setup, Git, project structure, .env
Week 2	Phase 3	FastAPI backend, auth, database connections
Week 3	Phase 4-5	RAG pipeline + report parser working

Week 4	Phase 6	Diet engine + 7-day plan generation
Week 5	Phase 7-8	ML models, symptom tracker, medication module
Week 6	Phase 9	Full React frontend with all pages
Week 7	Phase 10-11	Database schemas + deployment to Render + Vercel
Week 8	Phase 12-13	Datasets, testing, demo video, documentation

Final Tips for a Fantastic Project

1. Always keep venv active when working on backend. 2. Commit code every day, even small changes. 3. Test each API endpoint in Swagger before building the frontend for it. 4. Use Google Colab for ML model training — it is free and has GPU access. 5. The medical disclaimer is NOT optional — include it on every page. 6. Start with the RAG chatbot first — it is the most impressive feature for demonstrations.

Good luck building CancerCare AI!

You are building something that can genuinely help people. That matters.